# Robot Path Navigator: Pseudocode

**Group 9C, Iteration 2**

This is pseudocode for every function in the skeleton, grouped by owner. Each section names the file the function lives in. Where the logic already exists in `Group9FinalDraft1.cpp`, the pseudocode follows it, so you can port the code line by line.

Notation: ← means "set to". Rows and columns are stored 0-based and shown to the user 1-based.

---

## Program flow at a glance

```
None
START
  set up console
  print header
  create AppState (empty grid, no points, no runs)
  REPEAT
      show menu
      read choice 0-10
      run the one task for that choice
  UNTIL choice = 0
END
```

Typical use: 1 Configure grid → 2 Set A and B → 3 Generate obstacles → 4 Show grid → 5 Run → 6 Results

---

## Project Leader (T0, T11)

### `main()` (main.cpp)

```
None
FUNCTION main
    setupConsole()
    printHeader()
    state ← new AppState
    running ← true
```

```
    WHILE running
        printMenu()
        choice ← readInt("Choose an option", 0, 10)

        SWITCH choice
            1  → taskConfigureGrid(state)
            2  → taskSetPoints(state)
            3  → taskGenerateObstacles(state)
            4  → taskShowGrid(state)
            5  → taskRunSimulation(state)
            6  → taskShowResults(state)
            7  → taskHelp(state)
            8  → taskReset(state)
            9  → taskSaveLoad(state)
            10 → taskHistory(state)
            0  → print "Goodbye!"
                 running ← false
        END SWITCH
    END WHILE

    RETURN 0
END FUNCTION
```

## `printMenu()` (main.cpp)

None

```
FUNCTION printMenu
    print menu title line
    FOR each option 1 to 10, then 0
        print option number and task name
    print bottom line
END FUNCTION
```

## `readInt(prompt, lo, hi)` (input.cpp, already DONE)

None

```
FUNCTION readInt(prompt, lo, hi) RETURNS int
    LOOP forever
        print prompt and "[lo-hi]: "
        read a whole line of text
        IF input stream has ended THEN exit program

        try to read one integer from the line
        IF an integer was read
           AND nothing else is left on the line
```

```
           AND lo ≤ value ≤ hi
        THEN
            RETURN value
        ELSE
            print "Invalid input - enter a whole number from lo to hi"
    END LOOP
END FUNCTION
```

### `notImplemented(taskName)` (tasks.cpp)

```
None
FUNCTION notImplemented(taskName)
    print taskName + " - not implemented yet"
END FUNCTION
```

---

## Isaiah (T1, T2, T3, T9)

### Grid class (grid.cpp)

```
None
CONSTRUCTOR Grid
    side ← 0
    clear()
END

FUNCTION setSize(newSide)
    side ← newSide
    clear()
END

FUNCTION getSize() RETURNS int
    RETURN side
END

FUNCTION clear
    FOR r ← 0 TO MAX_SIDE-1
        FOR c ← 0 TO MAX_SIDE-1
            cells[r][c] ← EMPTY
END

FUNCTION inBounds(r, c) RETURNS bool
    RETURN 0 ≤ r < side AND 0 ≤ c < side
```

```
END

FUNCTION isFree(r, c) RETURNS bool
    RETURN inBounds(r, c) AND cells[r][c] ≠ OBSTACLE
END

FUNCTION getCell(r, c) RETURNS Cell
    RETURN cells[r][c]
END

FUNCTION markVisited(r, c)
    IF inBounds(r, c) AND cells[r][c] ≠ OBSTACLE
        cells[r][c] ← VISITED
END

FUNCTION clearVisited                       // keeps obstacles, wipes the old path
    FOR every cell (r, c) inside side x side
        IF cells[r][c] = VISITED THEN cells[r][c] ← EMPTY
END
```

## `placeObstacles(count, size, A, B, rng)` (grid.cpp)

None

```
FUNCTION placeObstacles(count, size, A, B, rng) RETURNS int
    clear()
    placed ← 0

    REPEAT count times
        REPEAT up to 200 tries
            horizontal ← random true/false
            IF horizontal
                r0 ← random 0 .. side-1
                c0 ← random 0 .. side-size
            ELSE
                r0 ← random 0 .. side-size
                c0 ← random 0 .. side-1

            ok ← true
            FOR k ← 0 TO size-1
                (r, c) ← (r0, c0+k) if horizontal, else (r0+k, c0)
                IF cells[r][c] ≠ EMPTY OR (r, c) = A OR (r, c) = B
                    ok ← false

            IF ok
                FOR k ← 0 TO size-1
                    mark that cell as OBSTACLE
```

```
                placed ← placed + 1
                stop trying (go to next obstacle)
        END REPEAT
    END REPEAT

    RETURN placed
END FUNCTION
```

## `pathExists(A, B)`: breadth-first search (grid.cpp)

None

```
FUNCTION pathExists(A, B) RETURNS bool
    seen[all] ← false
    queue ← [A]
    seen[A] ← true

    WHILE queue not empty
        p ← take front of queue
        IF p = B THEN RETURN true
        FOR each neighbour n of p (up, down, left, right)
            IF isFree(n) AND NOT seen[n]
                seen[n] ← true
                add n to back of queue

    RETURN false
END FUNCTION
```

## `save(filename)` and `load(filename)`: T9, optional (grid.cpp)

None

```
FUNCTION save(filename) RETURNS bool
    open filename for writing
    IF it could not be opened THEN RETURN false
    write side on the first line
    FOR each row r
        write one line: for each column, 1 if OBSTACLE else 0
    RETURN true
END

FUNCTION load(filename) RETURNS bool
    open filename for reading
    IF it could not be opened THEN RETURN false
    read newSide
    IF newSide < MIN_SIDE OR newSide > MAX_SIDE THEN RETURN false
    setSize(newSide)
```

```
    FOR each row r, each column c
        read value
        cells[r][c] ← OBSTACLE if value = 1 else EMPTY
    IF the file ended early THEN clear() and RETURN false
    RETURN true
END
```

## `readPoint(name, side)` (input.cpp)

None

```
FUNCTION readPoint(name, side) RETURNS Point
    print "Enter coordinates for point " + name
    p.r ← readInt("Row", 1, side) - 1
    p.c ← readInt("Column", 1, side) - 1
    RETURN p
END FUNCTION
```

## `taskConfigureGrid(state)`: T1 (tasks.cpp)

None

```
FUNCTION taskConfigureGrid(state)
    side ← readInt("Enter the grid side length", MIN_SIDE, MAX_SIDE)
    state.grid.setSize(side)
    obstacleSize ← side / 2                    // integer division
    print "Each obstacle will be obstacleSize cell(s) long"
    state.obstacleCount ← readInt("Enter the number of obstacles", 0, side-1)

    // the grid changed, so earlier points and obstacles are no longer valid
    state.A ← unset,  state.B ← unset
    state.obstaclesReady ← false
    print "Grid configured. Next: option 2 to set A and B."
END FUNCTION
```

## `taskSetPoints(state)`: T2 (tasks.cpp)

None

```
FUNCTION taskSetPoints(state)
    IF state.grid.getSize() = 0
        print "Configure the grid first (option 1)."
        RETURN

    side ← state.grid.getSize()
    state.A ← readPoint("A (start)", side)
```

```
    LOOP
        state.B ← readPoint("B (goal)", side)
        IF state.B ≠ state.A THEN stop looping
        print "Point B must be different from point A."
    END LOOP

    state.obstaclesReady ← false           // obstacles must avoid the new A and B
    print "Points set. Next: option 3 to generate obstacles."
END FUNCTION
```

## `taskGenerateObstacles(state)`: T3 (tasks.cpp)

None

```
FUNCTION taskGenerateObstacles(state)
    IF grid not configured OR A or B not set
        print "Set up the grid (1) and points (2) first."
        RETURN

    size  ← state.grid.getSize() / 2
    valid ← false
    REPEAT up to LAYOUT_ATTEMPTS times, while NOT valid
        placed ← state.grid.placeObstacles(state.obstacleCount, size, A, B,
state.rng)
        valid  ← (placed = state.obstacleCount) AND state.grid.pathExists(A, B)

    IF NOT valid
        print "Warning: could not place all obstacles with a guaranteed route."
        print "Using a layout with placed obstacle(s); B may be unreachable."

    state.obstaclesReady ← true
    print "Obstacles generated. Use option 4 to view or 5 to run."
END FUNCTION
```

## `taskSaveLoad(state)`: T9, optional (tasks.cpp)

None

```
FUNCTION taskSaveLoad(state)
    print "1. Save map   2. Load map   0. Back"
    choice ← readInt("Choose", 0, 2)
    IF choice = 0 THEN RETURN

    read filename from the user
    IF choice = 1
        IF grid not configured THEN print "Nothing to save." and RETURN
```

```
        IF state.grid.save(filename) THEN print "Saved." ELSE print "Could not
save."
    ELSE
        IF state.grid.load(filename)
            state.A ← unset, state.B ← unset
            state.obstaclesReady ← false
            print "Loaded. Set A and B with option 2."
        ELSE
            print "Could not load that file."
END FUNCTION
```

Design note: a loaded map already has its obstacles, but option 2 clears `obstaclesReady`. Decide how T2 and T3 should treat a loaded map. One option is an `AppState.mapLoaded` flag, so the user can skip option 3, and T2 rejects points that land on an obstacle.

---

## Robert (T5, T6, T10)

### Robot class (robot.cpp)

```
None
CONSTRUCTOR Robot
    pos ← unset, moves ← 0, distance ← 0, battery ← BATTERY_START
END

FUNCTION reset(start)
    pos ← start
    moves ← 0
    distance ← 0
    battery ← BATTERY_START
END

FUNCTION move(dr, dc)
    pos.r ← pos.r + dr
    pos.c ← pos.c + dc
    moves ← moves + 1
    distance ← distance + METERS_PER_MOVE
    battery ← battery - BATTERY_PER_MOVE
END

FUNCTION hasBattery() RETURNS bool
    RETURN battery ≥ BATTERY_PER_MOVE
END
```

```
getPosition / getMoves / getDistance / getBattery → RETURN the matching field
```

### Navigator constructor (navigator.cpp, already written)

```
None
CONSTRUCTOR Navigator(grid, start, goal, rng)
    store grid (by reference), A ← start, B ← goal, rng
    phase ← HORIZONTAL
    lane ← start.r
    detouring ← false, lastForward ← false, retreating ← false
    sideDir ← 1
    outcome ← RUNNING
    robot.reset(start)
END
```

### Private helper: `moveRobot(dr, dc)` (add to the Navigator class)

```
None
FUNCTION moveRobot(dr, dc)
    grid.markVisited(robot position)       // leave a trail where the robot was
    robot.move(dr, dc)
END
```

The draft does this in a lambda. `Robot` doesn't know about the grid, so `Navigator` marks the cell before moving.

### `run()`: T5 (navigator.cpp)

```
None
FUNCTION run() RETURNS Outcome
    render(grid, robot position, A, B, 0, 0, battery, "Robot powered on at point
A")
    pauseMs(STEP_DELAY_MS)

    WHILE step() returns true
        pauseMs(STEP_DELAY_MS)

    RETURN outcome
END FUNCTION
```

## `step()`: one move of the simulation (navigator.cpp)

```
None
FUNCTION step() RETURNS bool               // true = keep going, false = run is
over
    pos ← robot.getPosition()
    IF pos = B
        outcome ← REACHED
        RETURN false

    note ← ""
    // ---- switch axis when lined up with B ----
    IF phase = HORIZONTAL AND pos.c = B.c
        phase ← VERTICAL, lane ← pos.c
        detouring, lastForward, retreating ← false
        note ← "Lined up with B - switching to vertical. "
    ELSE IF phase = VERTICAL AND pos.r = B.r
        phase ← HORIZONTAL, lane ← pos.r
        detouring, lastForward, retreating ← false
        note ← "Re-aligning horizontally. "

    // ---- battery check ----
    IF NOT robot.hasBattery()
        outcome ← BATTERY_DEAD
        RETURN false

    // ---- directions ----
    IF phase = HORIZONTAL
        forward ← (0, +1 if B.c > pos.c else -1)
        perp    ← (1, 0)                       // side-steps go up/down
        perpOff ← pos.r - lane,  perpToGoal ← B.r - pos.r
    ELSE
        forward ← (+1 if B.r > pos.r else -1, 0)
        perp    ← (0, 1)                       // side-steps go left/right
        perpOff ← pos.c - lane,  perpToGoal ← B.c - pos.c

    moved ← false

    // ---- normal travel ----
    IF NOT detouring
        IF cell ahead is free
            moveRobot(forward), lastForward ← true, moved ← true
            status ← "Path clear - moving <dir>"
        ELSE
            detouring ← true, lastForward ← false
            sideDir ← +1 if perpToGoal > 0, -1 if perpToGoal < 0, else random ±1
            note ← note + "Obstacle ahead! "
```

```
    // ---- detour, in priority order ----
    IF NOT moved
        backDir ← -1 if perpOff > 0 else +1          // toward the original lane
        side    ← sideDir × perp

        IF perpOff ≠ 0 AND lastForward AND cell toward lane is free
            moveRobot(backDir × perp), lastForward ← false
            IF |perpOff| = 1
                detouring ← false
                status ← "Obstacle cleared - back on original path"
            ELSE status ← "Stepping toward original path"

        ELSE IF NOT retreating AND cell ahead is free
            moveRobot(forward), lastForward ← true
            IF perpOff = 0 THEN detouring ← false, status ← "Path clear"
            ELSE status ← "Moving alongside obstacle"

        ELSE IF cell at side is free
            moveRobot(side), lastForward ← false, retreating ← false
            status ← "Blocked - side-stepping"

        ELSE IF cell at opposite side is free
            sideDir ← -sideDir
            moveRobot(-side), lastForward ← false, retreating ← false
            status ← "That side is blocked - side-stepping"

        ELSE IF cell behind is free
            moveRobot(-forward), lastForward ← false, retreating ← true
            status ← "Dead end - backing up"

        ELSE
            outcome ← TRAPPED
            status ← "No possible moves"

    render(grid, robot position, A, B, moves, distance, battery, note + status)
    RETURN outcome = RUNNING
END FUNCTION
```

## `getOutcome()` and `getResult()` (navigator.cpp, already written)

```
None
FUNCTION getOutcome() RETURNS Outcome
    RETURN outcome

FUNCTION getResult() RETURNS RunResult
    RETURN { outcome, robot.getMoves(), robot.getDistance(), robot.getBattery() }
```

## `printResults(result)`: T6 (navigator.cpp)

```
None
FUNCTION printResults(result)
    print divider
    SWITCH result.outcome
        REACHED      → print "SUCCESS! The robot reached point B."
        BATTERY_DEAD → print "ROBOT DIED - battery depleted. Failed to navigate to
point B."
        TRAPPED      → print "ROBOT STUCK - boxed in by obstacles. Failed to
navigate to point B."
    print "Total moves       : " result.moves
    print "Distance traveled : " result.distance " m"
    print "Battery remaining : " result.battery "%"
    print divider
END FUNCTION
```

## `taskRunSimulation(state)`: T5 (tasks.cpp)

```
None
FUNCTION taskRunSimulation(state)
    IF NOT state.obstaclesReady
        print "Finish options 1, 2 and 3 first."
        RETURN

    state.grid.clearVisited()                  // wipe the path from any earlier
run
    print "Press ENTER to begin the simulation..."
    wait for ENTER

    nav ← Navigator(state.grid, state.A, state.B, state.rng)
    nav.run()

    state.lastRun ← nav.getResult()
    add state.lastRun to state.history
    state.hasRun ← true
    printResults(state.lastRun)
END FUNCTION
```

## `taskShowResults(state)`: T6 (tasks.cpp)

```
None
FUNCTION taskShowResults(state)
    IF NOT state.hasRun
        print "No simulation has been run yet (option 5)."
```

```
        RETURN
    printResults(state.lastRun)
END FUNCTION
```

### `taskHistory(state)`: T10, optional (tasks.cpp)

None

```
FUNCTION taskHistory(state)
    IF state.history is empty
        print "No runs yet."
        RETURN

    successes ← 0, totalMoves ← 0, totalBattery ← 0
    print table header: Run | Outcome | Moves | Battery
    FOR i ← 0 TO history.size-1
        run ← history[i]
        print i+1, outcome name, run.moves, run.battery
        IF run.outcome = REACHED THEN successes ← successes + 1
        totalMoves ← totalMoves + run.moves
        totalBattery ← totalBattery + run.battery

    n ← history.size
    print "Runs: n   Successes: successes   Success rate: successes × 100 / n %"
    print "Average moves: totalMoves / n   Average battery left: totalBattery / n
%"
END FUNCTION
```

---

## Wyatt (T4, T7, T8)

### Console helpers (display.cpp)

None

```
FUNCTION setupConsole
    IF running on Windows
        set console output to UTF-8
        turn on ANSI escape codes (virtual terminal processing)
END

FUNCTION clearScreen
    print ANSI code "clear screen and move cursor to top-left"
END
```

```
FUNCTION pauseMs(ms)
    sleep for ms milliseconds
END
```

## `printHeader()` and `printLegend()`: T7 (display.cpp)

None

```
FUNCTION printHeader
    print title banner "ROBOT PATH NAVIGATOR"
    print program name and authors
    print description paragraph (A to B, obstacles, 1 m and 2% battery per move,
50 moves max)
    print divider
END

FUNCTION printLegend
    print "Legend:"
    print SYM_ROBOT    "Robot (current position)"
    print SYM_START    "Point A (start)"
    print SYM_GOAL     "Point B (goal)"
    print SYM_OBSTACLE "Obstacle"
    print SYM_PATH     "Path already travelled"
    print SYM_EMPTY    "Open space"
    print "Coordinates are entered as (row, column), starting at 1 in the top-left
corner."
END
```

## `cellSymbol(grid, r, c, robot, A, B)`: T4 (display.cpp)

None

```
FUNCTION cellSymbol(grid, r, c, robot, A, B) RETURNS string
    IF (r, c) = robot THEN RETURN SYM_ROBOT        // checked first so it draws on
top
    IF (r, c) = A     THEN RETURN SYM_START
    IF (r, c) = B     THEN RETURN SYM_GOAL
    SWITCH grid.getCell(r, c)
        OBSTACLE → RETURN SYM_OBSTACLE
        VISITED  → RETURN SYM_PATH
        default  → RETURN SYM_EMPTY
END FUNCTION
```

## `render(grid, robot, A, B, moves, distance, battery, status)`: T4 (display.cpp)

```
None
FUNCTION render(grid, robot, A, B, moves, distance, battery, status)
    clearScreen()
    print title line
    side ← grid.getSize()

    print column numbers 1..side
    print top border
    FOR r ← 0 TO side-1
        print row number r+1 and "|"
        FOR c ← 0 TO side-1
            print " " + cellSymbol(grid, r, c, robot, A, B)
        print " |"
    print bottom border

    filled ← battery / 5                        // 20-segment battery bar
    print "Moves    : " moves
    print "Distance : " distance " m"
    print "Battery  : [" filled × "=" + (20 - filled) × " " "] " battery "%"
    print "Position : (robot.r+1, robot.c+1)    Goal B: (B.r+1, B.c+1)"
    print "Status   : " status
    print one-line legend
END FUNCTION
```

## `taskShowGrid(state)`: T4 (tasks.cpp)

```
None
FUNCTION taskShowGrid(state)
    IF state.grid.getSize() = 0
        print "Configure the grid first (option 1)."
        RETURN
    render(state.grid, state.A, state.A, state.B, 0, 0, BATTERY_START, "Map
preview")
    // robot is drawn at A because it hasn't moved yet
END FUNCTION
```

## `taskHelp(state)`: T7 (tasks.cpp)

```
None
FUNCTION taskHelp(state)
    printHeader()
```

```
    printLegend()
    print "Typical order: 1 → 2 → 3 → 4 → 5 → 6"
END FUNCTION
```

### `taskReset(state)`: T8 (tasks.cpp)

```
None
FUNCTION taskReset(state)
    answer ← readInt("Clear the grid, points and last run? 1 = yes, 0 = no", 0, 1)
    IF answer = 0 THEN RETURN

    state.grid.setSize(0)                        // also clears every cell
    state.obstacleCount ← 0
    state.A ← unset, state.B ← unset
    state.obstaclesReady ← false
    state.hasRun ← false
    // state.history is kept so T10 still shows the whole session
    print "Reset complete. Start again with option 1."
END FUNCTION
```

---

## Who depends on whom

| Function | Calls functions owned by |
|---|---|
| `taskGenerateObstacles` (Isaiah) | Isaiah only |
| `taskShowGrid` (Wyatt) | Wyatt's `render`, which reads Isaiah's `Grid` |
| `taskRunSimulation` (Robert) | Robert's `Navigator`, Isaiah's `Grid`, Wyatt's `render` / `pauseMs` |
| `Navigator::step` (Robert) | Isaiah's `isFree` / `markVisited`, Wyatt's `render` |

The full simulation (option 5) only works once all three people have filled in their parts, so tell the group chat when your functions are done.